Index

SR. NO.	TITLE	SIGN
1	**Practical 1** Data Partitioning and Visualization on 'ToyotaCorolla' dataset	
2	**Practical 2** Time series analysis on 'ApplianceShipments' dataset	
3	**Practical 3** Aggregation with boxplots on 'LaptopSalesJanuary2008' dataset	
4	**Practical 4** Naïve Bayes Classifier on 'Iris' dataset	
5	**Practical 5** K – Nearest – Neighbor model on 'Iris' dataset	
6	**Practical 6** Decision Tree on dataset 'Iris' dataset	
7	**Practical 7** DBScan clustering Model on Iris dataset	
8	**Practical 8** Hierarchical Clustering Model on 'mtcars' dataset	

Practical 1

The dataset ToyotaCorolla.csv contains data on used cars on sale during the late summer of 2004 in the Netherlands. It has 1436 records containing details on attributes, including Price, Age, Kilometers, HP, and other specifications.

a) Show the different subsets of the whole dataset

b) Find mean of and summary statistics for the dataset

c) The dataset has two categorical attributes, Fuel Type and Metallic. Convert these to binary variables so that categorical data is transformed into dummies.

d) Prepare the dataset for data mining techniques of supervised learning by creating partitions in R. Select all the variables and use default values for the random seed and partitioning percentages for training (50%), validation (30%), and test (20%) sets. Describe the roles that these partitions will play in modeling.

e) Explore the data using the data visualization capabilities of R. Which of the pairs among the variables seem to be correlated?

SOLUTION

The dataset used is: TototaCorolla.csv

Description of the attributes of the dataset

- Price : offer price in EUROS
- Age : in years
- KM : Accumulated kilometers
- FuelType : Fuel type(Petrol, Diesel CNG) categorical
- HP : Horse Power(HP- unit of measurement of power)
- MetColor : Metallic color? (yes=1, no=0)
- Automatic : Automatic? (Yes=1, No=0)
- CC : Cylinder volume in cubic centimeters
- Doors : Number of doors
- Weight : Weight in Kilograms

a) Show the different subsets of the whole dataset

```
> # Practical 1
> setwd("C:\\MSc Data Science\\Data Mining")
> # Exercise 1
> # Read csv from disk along with column names
> df = read.csv("ToyottaCorolla.csv", header = TRUE)
> # Display first five records
> head(df,5)
  Price Age    KM FuelType HP MetColor Automatic   CC Doors Weight
1 13500  23 46986   Diesel 90        1         0 2000     3   1165
2 13750  23 72937   Diesel 90        1         0 2000     3   1165
3 13950  24 41711   Diesel 90        1         0 2000     3   1165
4 14950  26 48000   Diesel 90        0         0 2000     3   1165
5 13750  30 38500   Diesel 90        0         0 2000     3   1170
> # Display the dimension of the dataframe
> dim(df)
[1] 1436   10
> # Display the column names of the dataframe
> names(df)
 [1] "Price"     "Age"       "KM"        "FuelType"  "HP"        "MetColor"  "Automatic" "CC"        "Doors"     "Weight"
> # Viewing the contents dataframe
> View(df)
> # Subsetting dataframe
> df[1:4,2]
[1] 23 23 24 26
> df[1:5,]
  Price Age    KM FuelType HP MetColor Automatic   CC Doors Weight
1 13500  23 46986   Diesel 90        1         0 2000     3   1165
2 13750  23 72937   Diesel 90        1         0 2000     3   1165
3 13950  24 41711   Diesel 90        1         0 2000     3   1165
4 14950  26 48000   Diesel 90        0         0 2000     3   1165
5 13750  30 38500   Diesel 90        0         0 2000     3   1170
> df[1:4, c(1,3)]
  Price    KM
1 13500 46986
2 13750 72937
3 13950 41711
4 14950 48000
> # Displaying specific column
> head(df$Age,5)
[1] 23 23 24 26 30
> # Displaying the length of specified column
> length(df$Age)
[1] 1436
> # Creating a subset using subset function
> df_subset = subset(df, select = 1:5)
> head(df_subset,5)
  Price Age    KM FuelType HP
1 13500  23 46986   Diesel 90
2 13750  23 72937   Diesel 90
3 13950  24 41711   Diesel 90
4 14950  26 48000   Diesel 90
5 13750  30 38500   Diesel 90
```

b) Find mean of and summary statistics for the dataset

```
> # Exercise 2
> # Calculate and display Mean
> mean(df $Price)
[1] 10730.82
> # Display Summary of the specified column
> summary(df$Price)
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max. 
   4350    8450    9900   10731   11950   32500 
> # Summary of dataframe
> summary(df)
     Price           Age              KM            FuelType          HP           MetColor        Automatic      
 Min.   : 4350   Min.   : 1.00   Min.   :     1   CNG   :  17   Min.   : 69.0   Min.   :0.0000   Min.   :0.00000  
 1st Qu.: 8450   1st Qu.:44.00   1st Qu.: 43000   Diesel: 155   1st Qu.: 90.0   1st Qu.:0.0000   1st Qu.:0.00000  
 Median : 9900   Median :61.00   Median : 63390   Petrol:1264   Median :110.0   Median :1.0000   Median :0.00000  
 Mean   :10731   Mean   :55.95   Mean   : 68533                 Mean   :101.5   Mean   :0.6748   Mean   :0.05571  
 3rd Qu.:11950   3rd Qu.:70.00   3rd Qu.: 87021                 3rd Qu.:110.0   3rd Qu.:1.0000   3rd Qu.:0.00000  
 Max.   :32500   Max.   :80.00   Max.   :243000                 Max.   :192.0   Max.   :1.0000   Max.   :1.00000  
       CC            Doors           Weight      
 Min.   :1300   Min.   :2.000   Min.   :1000  
 1st Qu.:1400   1st Qu.:3.000   1st Qu.:1040  
 Median :1600   Median :4.000   Median :1070  
 Mean   :1567   Mean   :4.033   Mean   :1072  
 3rd Qu.:1600   3rd Qu.:5.000   3rd Qu.:1085  
 Max.   :2000   Max.   :5.000   Max.   :1615  
> 
```

c) The dataset has two categorical attributes, Fuel Type and Metallic. Convert these to binary variables so that categorical data is transformed into dummies

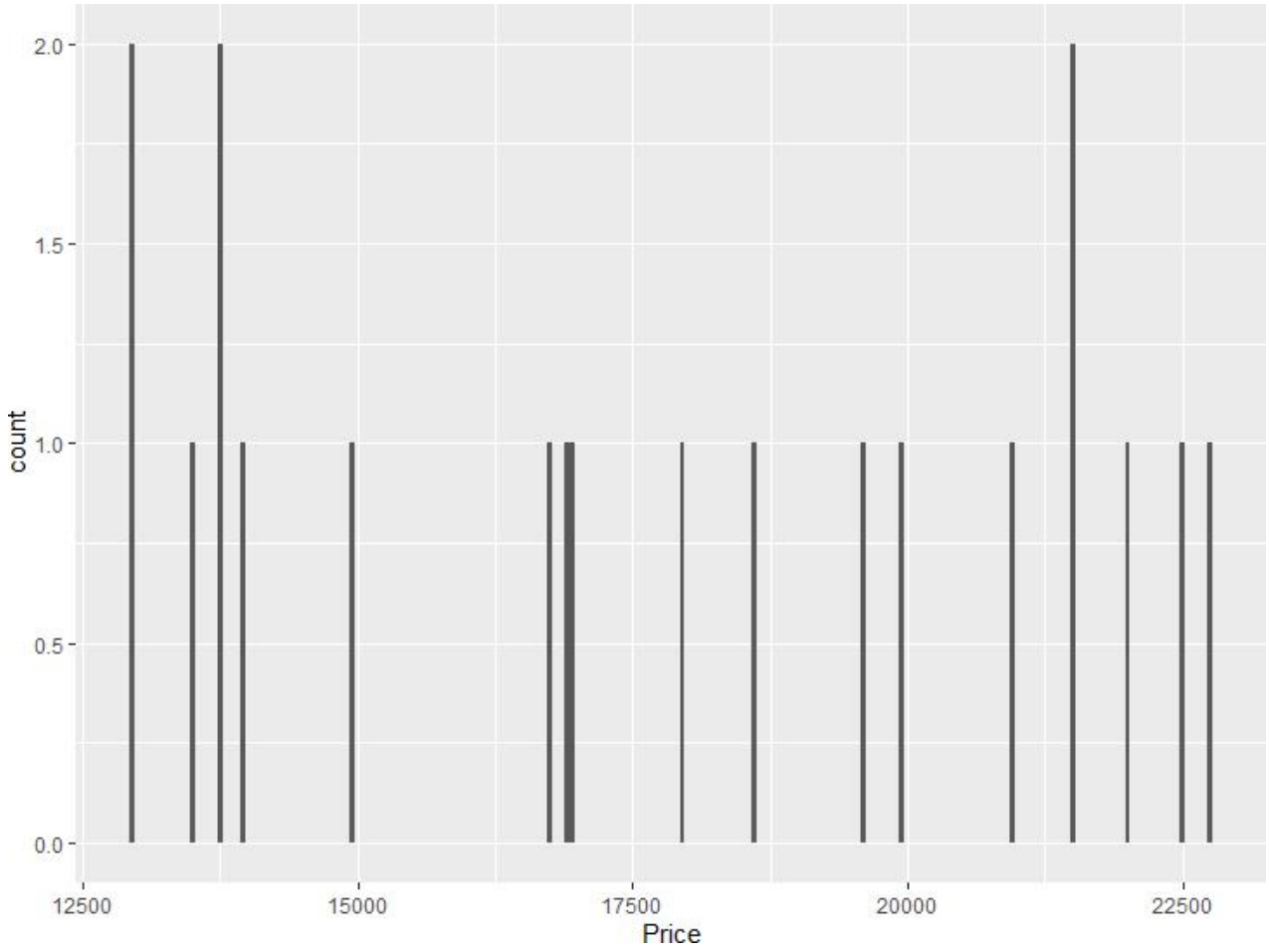
```
> # Exercise 3
> # Convert categorical to binary
> xtotal = model.matrix(~0 + FuelType + MetColor,data = df)
> xtotal = as.data.frame(xtotal)
> head(xtotal)
  FuelTypeCNG FuelTypeDiesel FuelTypePetrol MetColor
1           0              1              0        1
2           0              1              0        1
3           0              1              0        1
4           0              1              0        0
5           0              1              0        0
6           0              1              0        0
> 
```

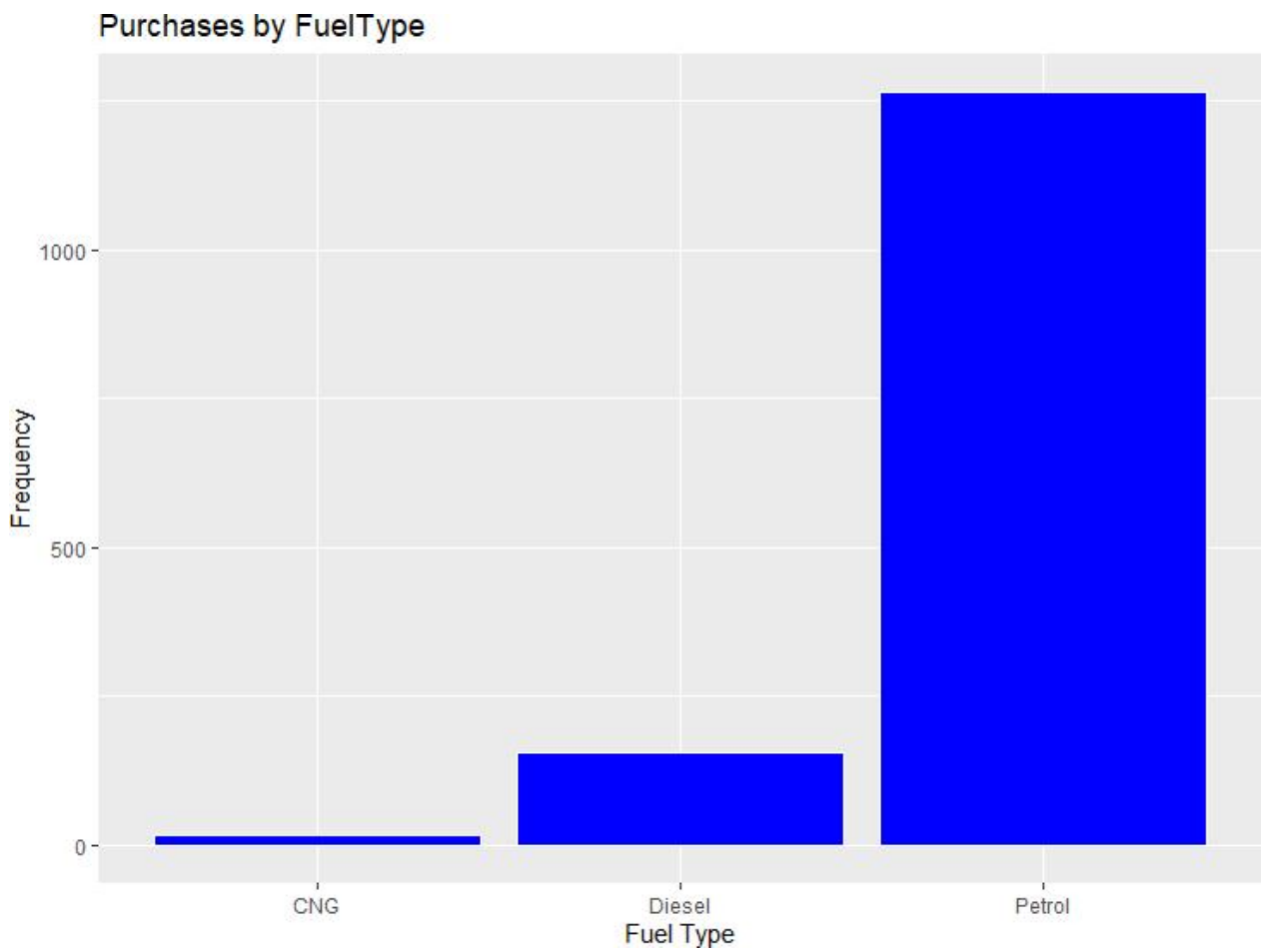
d) Prepare the dataset for data mining techniques of supervised learning by creating partitions in R. Select all the variables and use default values for the random seed and partitioning percentages for training (50%), validation (30%), and test (20%) sets. Describe the roles that these partitions will play in modeling.

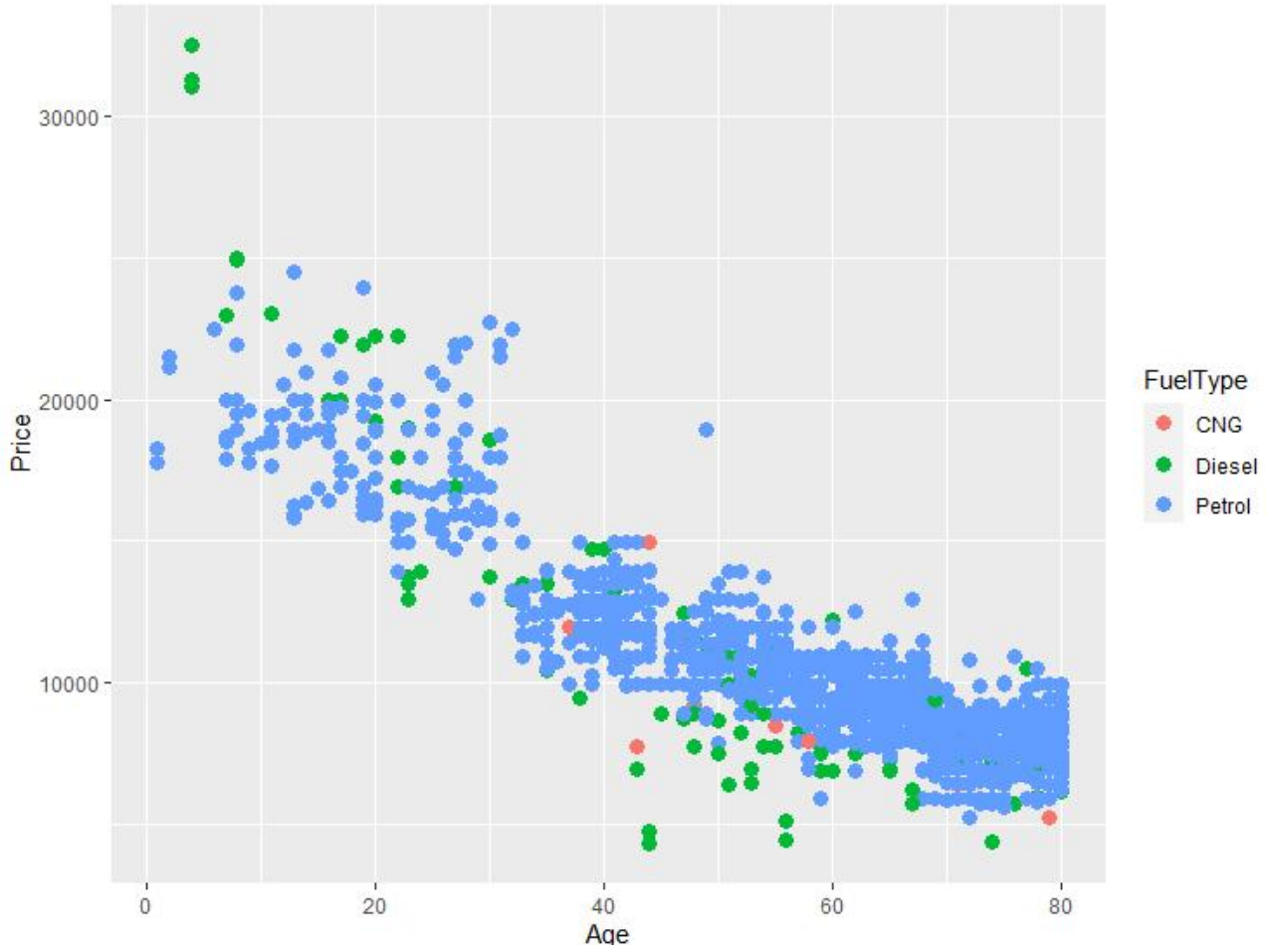
```
> # Exercise 4
> # Setting seed value for randomization
> set.seed(1)
> # Partitioning dataframe
> train.rows = sample(rownames(df),dim(df)[1]*0.5)
> valid.rows = sample(setdiff(rownames(df),train.rows),dim(df)[1]*0.3)
> test.rows = setdiff(rownames(df),union(train.rows,valid.rows))
> # Selecting rows from dataframe
> train.data = df[train.rows, ]
> valid.data = df[valid.rows, ]
> test.data = df[test.rows, ]
> # Viewing the dataset
> head(train.data)
     Price Age     KM FuelType  HP MetColor Automatic   CC Doors Weight
1017  9250  65  35283   Petrol  86        1         0 1300     3   1015
679   9895  68 102494   Petrol 110        0         0 1600     5   1090
129  17950  17  33740   Petrol  97        1         0 1400     5   1135
930   9995  57  55844   Petrol  86        1         0 1300     5   1045
471  10900  50  65471   Petrol  97        1         0 1400     5   1060
299  13995  35  41000   Petrol 110        1         0 1600     5   1075
> head(valid.data)
    Price Age    KM FuelType  HP MetColor Automatic   CC Doors Weight
274 13450  34 48011   Petrol 110        1         0 1600     4   1030
965 10950  65 49942   Petrol 110        0         0 1600     3   1065
477 10250  54 63792   Petrol 110        1         0 1600     5   1075
558 13750  54 40325   Petrol  86        0         0 1300     5   1045
942  9250  61 54000   Petrol 110        1         0 1600     3   1055
47  17950  27 79375   Diesel  90        1         0 2000     5   1255
> head(test.data)
   Price Age    KM FuelType  HP MetColor Automatic   CC Doors Weight
2  13750  23 72937   Diesel  90        1         0 2000     3   1165
4  14950  26 48000   Diesel  90        0         0 2000     3   1165
7  16900  27 94612   Diesel  90        1         0 2000     3   1245
10 12950  23 71138   Diesel  69        0         0 1900     3   1105
21 15950  30 67660   Petrol 110        1         0 1600     3   1105
23 15950  28 56349   Petrol 110        1         0 1600     3   1120
>
```

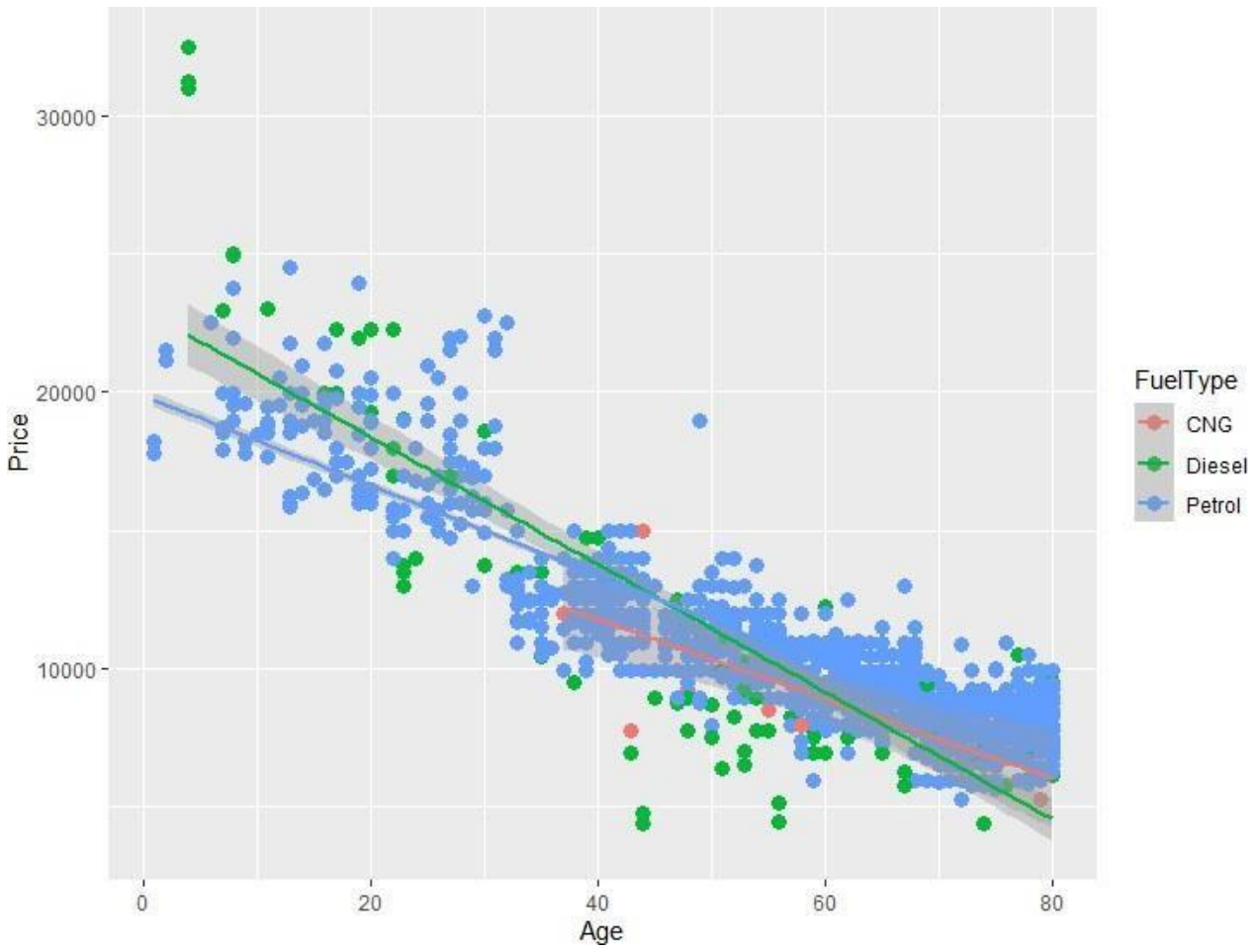
e) Explore the data using the data visualization capabilities of R. Which of the pairs among the variables seem to be correlated?

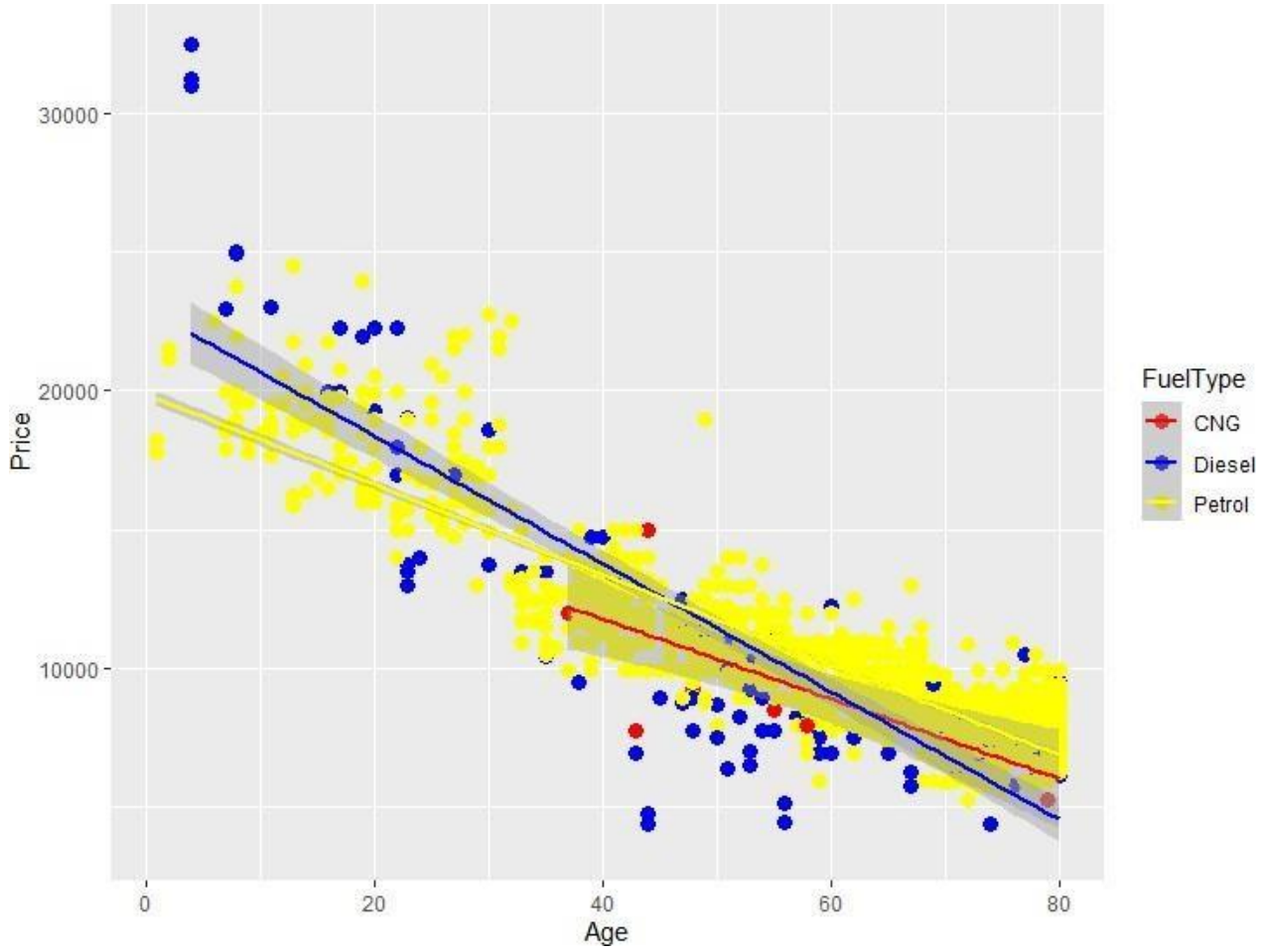
```
> ggplot(df,aes(x=Age,y=Price)) + geom_point(color="blue",size=3,alpha=1)
> # Give color as per the Fuel Type
> ggplot(df,aes(x=Age,y=Price,color=FuelType)) + geom_point(size=3)
> # Add a line of Best Fit
> # lm indicates linear model
> ggplot(df,aes(x=Age,y=Price,color=FuelType)) + geom_point(size=3) + geom_smooth(method = "lm")
`geom_smooth()` using formula 'y ~ x'
> # Choose custom colors
> ggplot(df,aes(x=Age,y=Price,color=FuelType)) + geom_point(size=3) + geom_smooth(method = "lm") + scale_colour_manual(values = c("red","blu
e","yellow"))
`geom_smooth()` using formula 'y ~ x'
> # Add a line of Best Fit
> # lm indicates linear model
> ggplot(df,aes(x=Age,y=Price,color=FuelType)) + geom_point(size=3) + geom_smooth(method = "lm")
`geom_smooth()` using formula 'y ~ x'
> # Exercise 5
> # Importing library for data visualization
> library(ggplot2)
> # Subsetting data
> df_sub = df[1:20,]
> # Bar plot
> ggplot(df_sub,aes(x=Price)) + geom_bar()
> ggplot(df,aes(x=FuelType)) + geom_bar(fill="blue", color="white") + labs(x="Fuel Type",y="Frequency",title ="Purchases by FuelType")
> # Scatter plot
> # visualize Correlation between Age and Price of the car
> ggplot(df,aes(x=Age,y=Price)) + geom_point(color="blue",size=3,alpha=1)
> # Give color as per the Fuel Type
> ggplot(df,aes(x=Age,y=Price,color=FuelType)) + geom_point(size=3)
> # Add a line of Best Fit
> # lm indicates linear model
> ggplot(df,aes(x=Age,y=Price,color=FuelType)) + geom_point(size=3) + geom_smooth(method = "lm")
`geom_smooth()` using formula 'y ~ x'
> # Choose custom colors
> ggplot(df,aes(x=Age,y=Price,color=FuelType)) + geom_point(size=3) + geom_smooth(method = "lm") + scale_colour_manual(values = c("red","blu
e","yellow"))
`geom_smooth()` using formula 'y ~ x'
>
```


2.0
1.5
1.0
0.5
0.0
count
12500
15000
17500
20000
22500
Price


Purchases by FuelType
Frequency
1000
500
0
CNG
Diesel
Petrol
Fuel Type


30000
20000
10000
Price
0
20
40
60
80
Age
FuelType
CNG
Diesel
Petrol


30000
20000
10000
Price
0
20
40
60
80
Age
FuelType
CNG
Diesel
Petrol


30000
20000
10000
Price
0
20
40
60
80
Age
FuelType
CNG
Diesel
Petrol

Practical 2

The file ApplianceShipments.csv contains the series of quarterly shipments (in millions of dollars) of US household appliances between 1985 and 1989.

1. Create a well-formatted time plot of the data using R.
2. Using R, create one chart with four separate lines, one line for each of Q1, Q2, Q3 and Q4. In R, this can be achieved by generating a dataframe for each quarter Q1, Q2, Q3, Q4, and then plotting them as separate series on the line graph.
3. Using R, create a line graph of the series at a yearly aggregated level (i.e., the total shipments in each year).

Solution

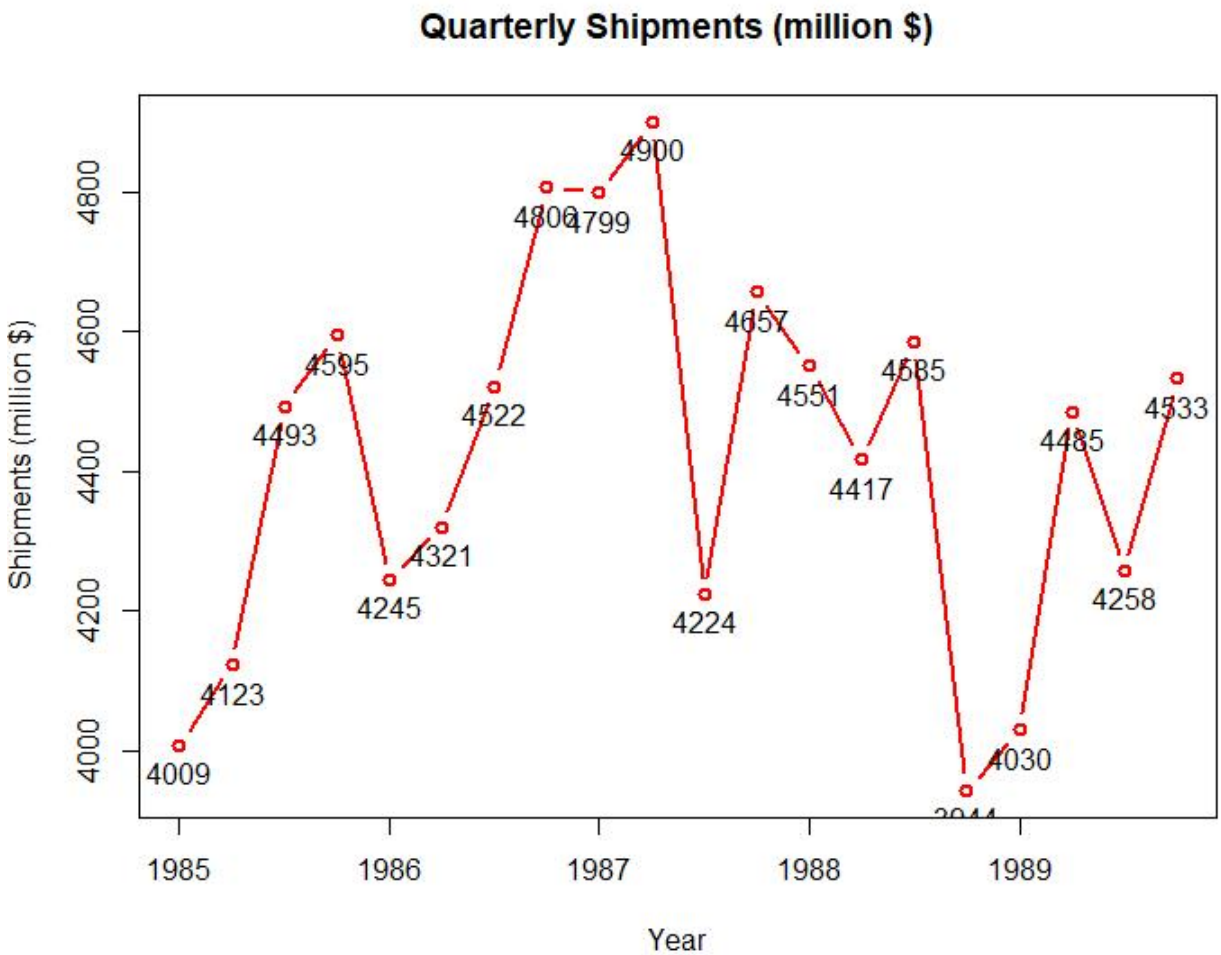
The dataset used is: ApplicanceShipments.csv

Description of the attributes of the dataset

- Quarter: Quarter of the Year in which shipment was done.
- Shipment: Value of shipments (in million dollars)

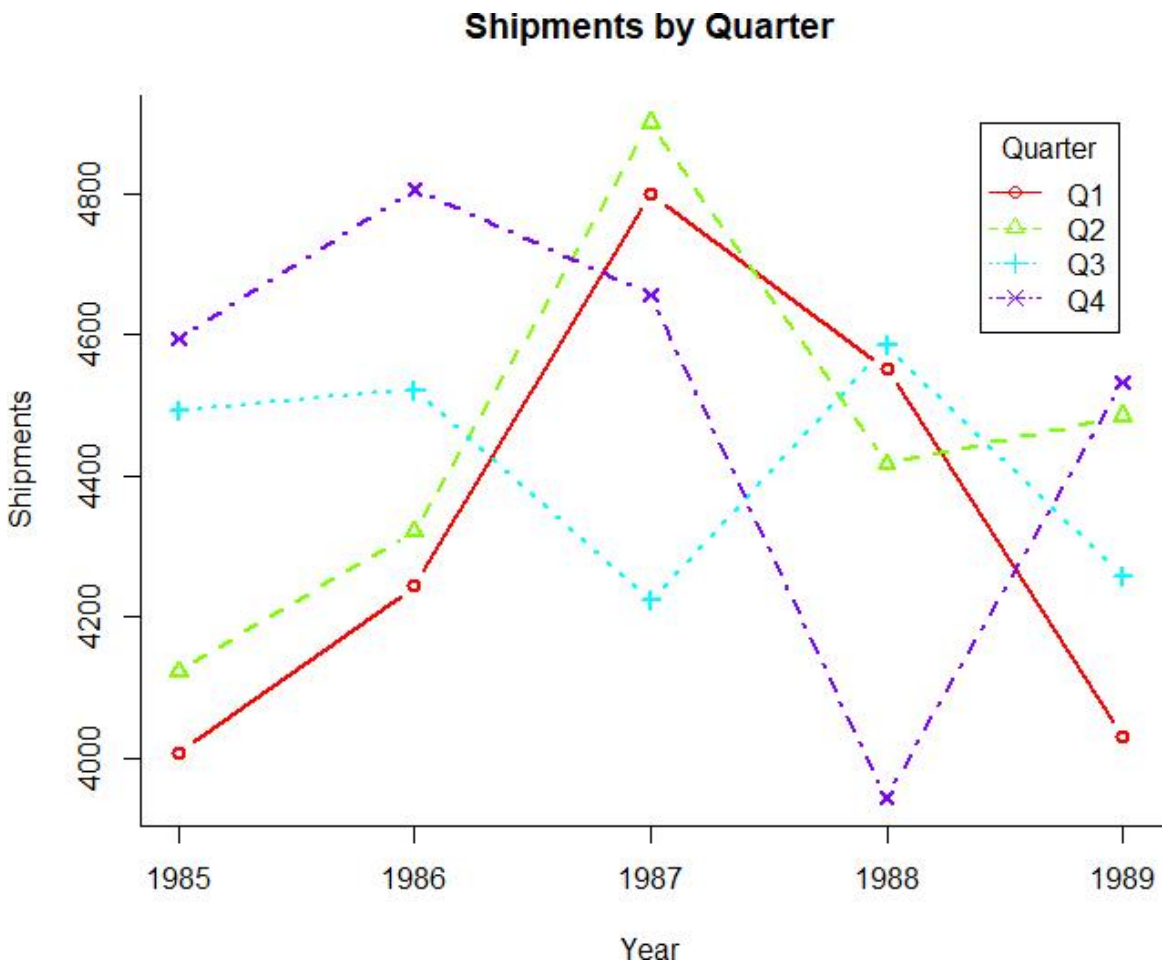
1. Create a well-formatted time plot of the data using R.

```
> # Practical 2
> setwd("C:\\MSc Data Science\\Data Mining\\Datasets")
> # Exercise 1
> # Reading dataset from disk
> dataset = read.csv("ApplianceShipments.csv", header = TRUE)
> # Viewing the content and summary of the dataset
> head(dataset)
  i..Quarter Shipments
1    Q1-1985      4009
2    Q1-1986      4123
3    Q1-1987      4493
4    Q1-1988      4595
5    Q1-1989      4245
6    Q2-1985      4321
> summary(dataset)
   i..Quarter   Shipments
 Q1-1985: 1   Min.   :3944
 Q1-1986: 1   1st Qu.:4240
 Q1-1987: 1   Median :4489
 Q1-1988: 1   Mean   :4425
 Q1-1989: 1   3rd Qu.:4588
 Q2-1985: 1   Max.   :4900
 (Other):14
> # Creating a time series from dataset
> time_series = ts(dataset$Shipments, start = c(1985,1), end = c(1989,4), frequency = 4)
> time_series
     Qtr1 Qtr2 Qtr3 Qtr4
1985 4009 4123 4493 4595
1986 4245 4321 4522 4806
1987 4799 4900 4224 4657
1988 4551 4417 4585 3944
1989 4030 4485 4258 4533
> # Plotting time plot
> plot(time_series, xlab="Year", ylab="Shipments (million $)", main="Quarterly Shipments (million $)", lwd=2.5, type="b", col="red" )
> # Adding labels (shipments) to the plot
> text(x=time_series, labels=time_series,pos=1, cex=1)
>
```



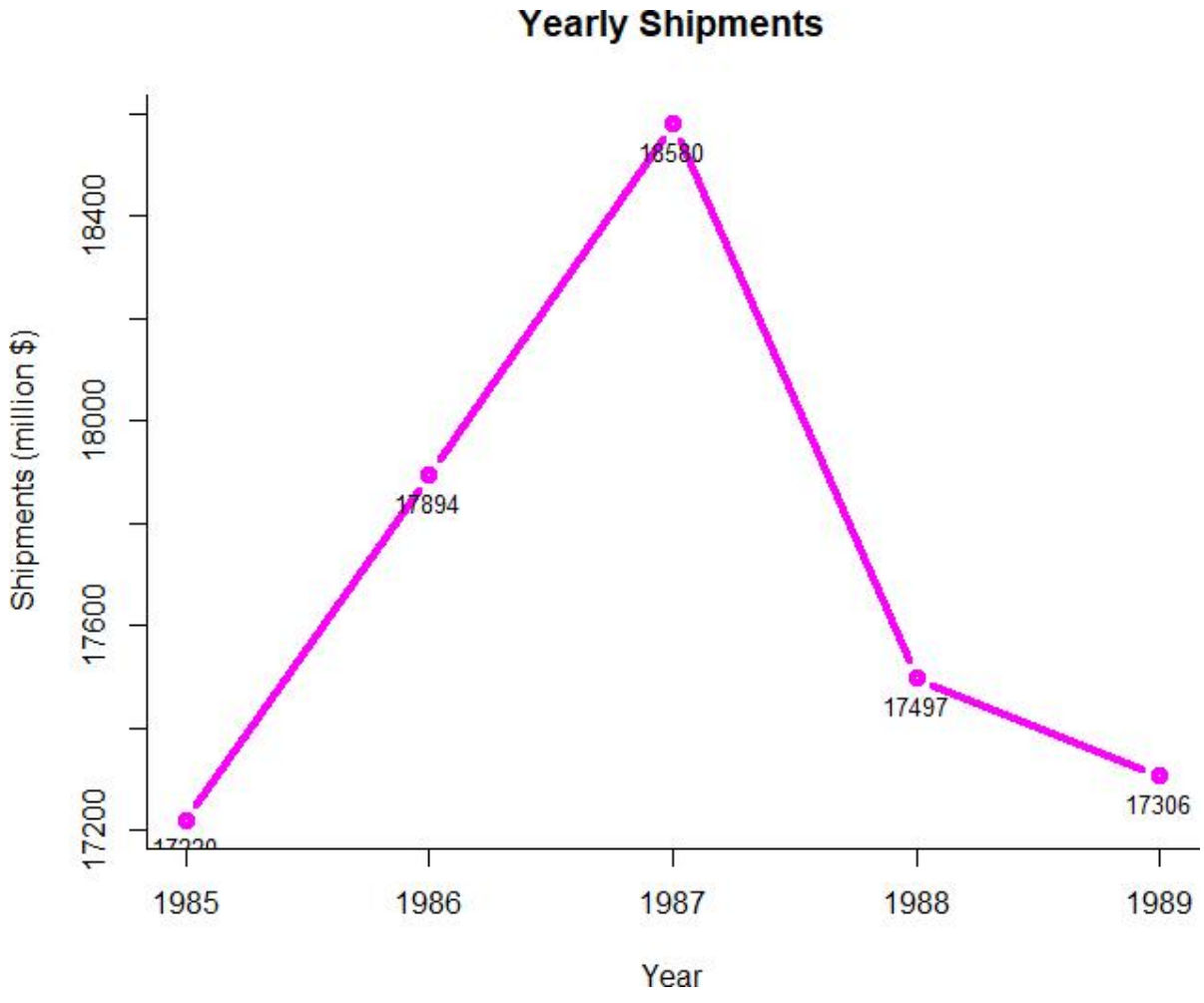
2. Using R, create of chart with four separate lines, one line for each of Q1, Q2, Q3 & Q4. In R, this can be achieved by generating a dataframe for each quarter and then plotting them as separate series on the line graph.

```
> # Exercise 2
> # Setting outer margins (bottom, left, top, right)
> par(oma = c(0,0,0,2))
> xrange = c(1,5) # 5 years, 1984-1989
> yrange = range(time_series)
> plot(xrange, yrange, main="Shipments by Quarter", xlab="Year", ylab="Shipments", type="n", bty="l", xaxt="n")
> # Labeling x axis with year
> axis(1, at=1:5, labels = 1985:1989)
> colors=rainbow(4)
> for(i in 1:4){
+   # Subsetting i'th quarter
+   current_quarter = subset(time_series, cycle(time_series)==i)
+   # Plotting line for the i'th  quarter
+   lines(current_quarter, type="b", lwd=2, lty=i, col=colors[i], pch=i)
+ }
> # Creating legend
> legend(4.4, 4900, c("Q1","Q2","Q3","Q4"), cex = 1, col=colors,pch=c(1:4), lty=c(1:4), title = "Quarter" )
>
```


Shipments by Quarter
Quarter
Q1
Q2
Q3
Q4
Shipments
4000
4200
4400
4600
4800
1985
1986
1987
1988
1989
Year

3. Using R, create a line graph of the series at a yearly aggregated level (i.e., the total shipments in each year).

```
> # Exercise 3
> # Aggregating each quarter to create yearly shipments
> yearly = aggregate(time_series, FUN=sum)
> # Plotting yearly shipments
> plot(yearly, xlab="Year", ylab="Shipments (million $)", main="Yearly Shipments", col="#FF00FF",lwd=4,  bty="l", type="b" )
> text(x=yearly, labels = yearly, pos=1, cex=0.8)
>
```



Practical 3

Laptop Sales at a London Computer Chain: The file LaptopSalesJanuary2008.csv contains data for all sales of laptops at a computer chain in London in January 2008. This is a subset of the full dataset that includes data for the entire year.

a. Create a bar chart, showing the average retail price by store. Which store has the highest average? Which has the lowest?
b. To better compare retail prices across stores, create side-by-side boxplots of retail price by store. Now compare the prices in the two stores from (a). Does there seem to be a difference between their price distributions?

Solution:

Dataset used: LaptopSales2008.csv

Description of the variables used:

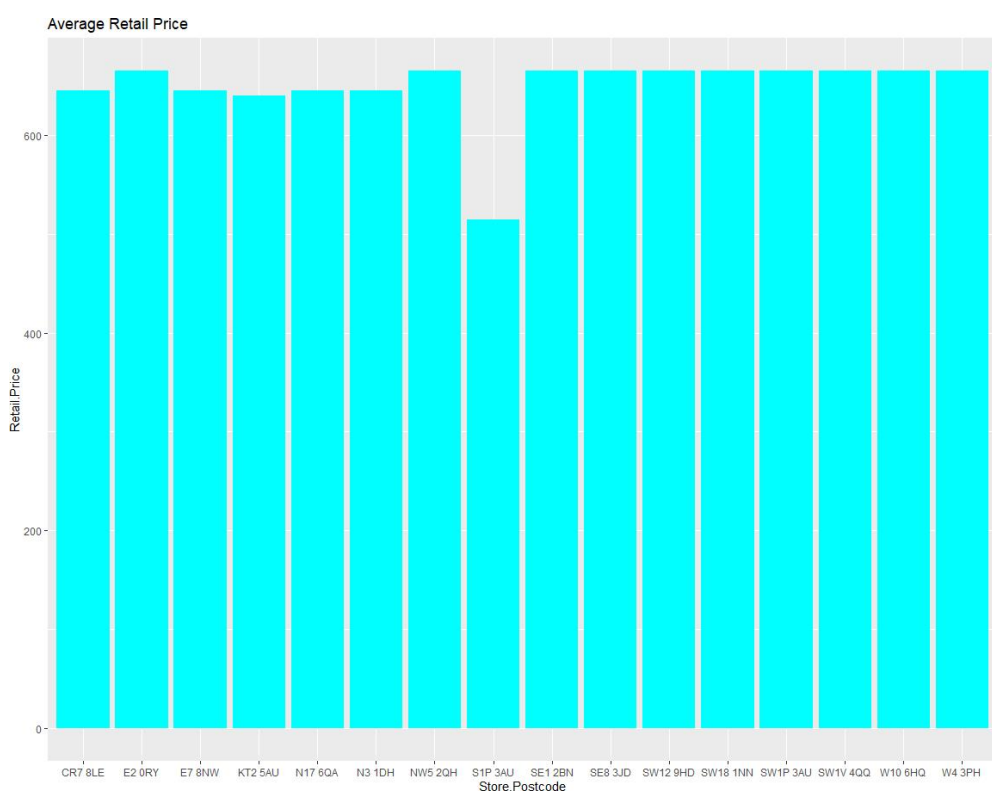
Date	Date and time of Purchase
Customer.Postcode	Postal Code of the Customer who purchased the Laptop
Store.Postcode	Postal Code of the Store from where the Laptop was purchased
Retail.Price	Price of the Laptop (in Dollars)
Screen.Size (Inches)	Size of the Laptop screen in inches
Battery.Life (Hours)	Life of the Battery (in hours)
Ram (GB)	Size of RAM (in GB)
Processor.Speeds (Ghz)	Processor Speed in GHz
Integrated.Wireless?	Is the Laptop integrated with wireless (Yes/ No)
HD.Size (GB)	Size of the Hard Disk (in GB)
CustomerStoreDistance ()	Distance between the Customer and the stores (in meters)

Exercise A:

Code:

```
> # Data Mining Practical 3
> # Set Current working directory
> setwd("C:\\MSc Data Science\\Data Mining\\Datasets")
> # Read data from disk
> laptopsales = read.csv("LaptopSalesJanuary2008.csv", header = TRUE)
> head(laptopsales)
          Date Customer.Postcode Store.Postcode Retail.Price Screen.Size..Inches. Battery.Life..Hours. RAM..GB. Processor.Speeds..GHz.
1 1/1/2008 0:01          EC4V 5BH        SE1 2BN          455                   15                    5        1                     2
2 1/1/2008 0:02           SW4 0JL       SW12 9HD          545                   15                    6        1                     2
3 1/1/2008 0:04          EC3V 1LR         E2 0RY          515                   15                    4        1                     2
4 1/1/2008 0:04          SW1P 3AU        SE1 2BN          395                   15                    5        1                     2
5 1/1/2008 0:06          EC4V 4EG       SW1V 4QQ          585                   15                    6        2                     2
6 1/1/2008 0:12           W1B 5PX       SW1V 4QQ          555                   15                    6        1                     2
  Integrated.Wireless. HD.Size..GB. CustomerStoreDistance
1                  Yes           80              2405.873
2                   No          300              2507.559
3                  Yes          300              3194.001
4                   No           40              4155.202
5                   No          120              3729.298
6                  Yes          120              2544.785
> # Import library
> library(ggplot2)
> # Exercise A
> # Plotting average retail price vs store location
> ggplot(laptopsales) + geom_bar(aes(Store.Postcode, Retail.Price), stat="summary", fun="max", fill="cyan" ) + ggtitle("Average Retail Price")
> # Calculating Average retail price for each store
> agg.data = aggregate(data = laptopsales, Retail.Price~Store.Postcode, mean)
> # Displaying store's average retail price in ascending order
> agg.data[order(agg.data$Retail.Price),]
   Store.Postcode Retail.Price
16         W4 3PH     481.0063
2          E2 0RY     483.1717
11       SW12 9HD     485.2957
8         S1P 3AU     486.2500
7         NW5 2QH     486.5805
9         SE1 2BN     486.6802
6          N3 1DH     487.3684
13       SW1P 3AU     488.5069
1         CR7 8LE     488.6190
14       SW1V 4QQ     489.3450
15        W10 6HQ     489.8667
10        SE8 3JD     492.1778
12       SW18 1NN     493.0389
4         KT2 5AU     493.9048
3          E7 8NW     494.3814
5         N17 6QA     494.6341
>
```

Plot(s):

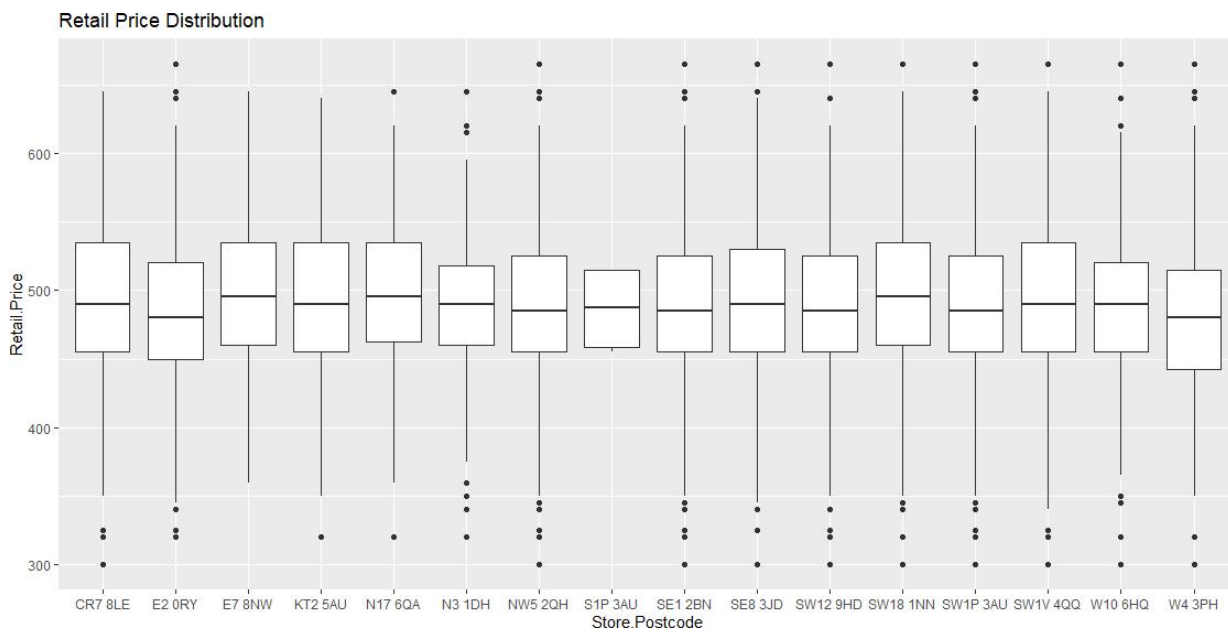


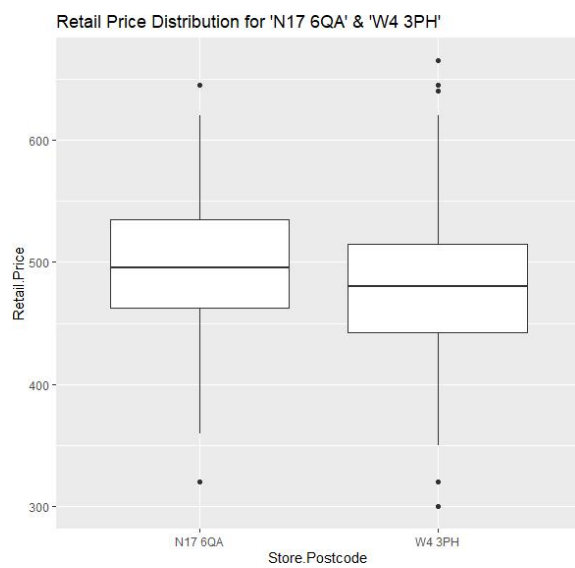
Exercise B:

Code:

```
> # Exercise B
> # Plotting Boxplot (Retail price distribution for each store)
> ggplot(laptopsales) + geom_boxplot(aes(Store.Postcode, Retail.Price)) + ggtitle("Retail Price Distribution")
> # Subsetting records of stores with Minimum and Maximum Average retail price respectively
> x = subset(laptopsales, Store.Postcode %in% c("W4 3PH", "N17 6QA"))
> # Plotting Boxplot (Retail price distribution for the Stores located at 'W4 3PH' and 'N17 6QA')
> ggplot(x) + geom_boxplot(aes(Store.Postcode, Retail.Price)) + ggtitle("Retail Price Distribution for 'N17 6QA' & 'W4 3PH'")
>
```

Plot(s):





Observations:

a. The store located at W439H has a lower Q1, Median and Q3 value compared to N176QA
b. The store located at W43PH has more outliers compared to the store located at N176QA

Practical 4

The dataset iris contains 50 samples from each of 3 species of Iris (Iris setosa, Iris virginica and Iris versicolor).

Exercise A

Split the data to training and test data. Build a Naïve Bayes Classifier for this data

Solution

```
> # Practical 4
> # Exercise A
> # Build Naive Bayes Classifier for iris dataset
> # Loading packages
> library(e1071)
> library(caTools)
>
> # Loading data
> data(iris)
> head(iris)
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1          5.1         3.5          1.4         0.2  setosa
2          4.9         3.0          1.4         0.2  setosa
3          4.7         3.2          1.3         0.2  setosa
4          4.6         3.1          1.5         0.2  setosa
5          5.0         3.6          1.4         0.2  setosa
6          5.4         3.9          1.7         0.4  setosa
>
> # Splitting data into train and test data
> split = sample.split(iris$Sepal.Length, SplitRatio = 0.6)
> train_class = subset(iris, split == TRUE)
> test_class = subset(iris, split == FALSE)
> # Building the Naive Bayes Classifier
> SpeciesClassifierNaive = naiveBayes(Species~., data = train_class)
> SpeciesClassifierNaive

Naive Bayes Classifier for Discrete Predictors

Call:
naiveBayes.default(x = X, y = Y, laplace = laplace)

A-priori probabilities:
Y
    setosa versicolor  virginica
 0.3440860  0.3010753  0.3548387

Conditional probabilities:
            Sepal.Length
Y               [,1]      [,2]
  setosa     4.987500 0.3452442
  versicolor 5.978571 0.5137784
  virginica  6.612121 0.6352940

            Sepal.Width
Y               [,1]      [,2]
  setosa     3.403125 0.3533110
  versicolor 2.860714 0.2910254
  virginica  2.987879 0.3276408

            Petal.Length
Y               [,1]      [,2]
  setosa     1.446875 0.1967221
  versicolor 4.317857 0.4682936
  virginica  5.524242 0.5420046

            Petal.Width
Y               [,1]       [,2]
  setosa     0.234375 0.07452809
  versicolor 1.378571 0.17502834
  virginica  2.039394 0.26450354

> # Predicting the test data
> predicted = predict(SpeciesClassifierNaive, newdata = test_class)
> predicted
 [1] setosa     setosa     setosa     setosa     setosa     setosa     setosa     setosa     setosa     setosa     setosa     setosa     setosa
[14] setosa     setosa     setosa     setosa     setosa     versicolor versicolor versicolor versicolor versicolor versicolor versicolor versicolor
[27] versicolor versicolor versicolor versicolor virginica  versicolor versicolor versicolor versicolor versicolor versicolor versicolor versicolor
[40] versicolor versicolor virginica  virginica  virginica  virginica  virginica  virginica  virginica  virginica  virginica  versicolor virginica
[53] virginica  virginica  virginica  virginica  virginica
Levels: setosa versicolor virginica
>
```

Exercise B

Build a confusion matrix and calculate the accuracy.

Solution

```
> # Exercise B
> # Compute the Confusion Matrix
> library(caret)
> # Predicted vs actual
> cm = table(test_class$Species, predicted)
> cm
            predicted
             setosa versicolor virginica
  setosa         18          0         0
  versicolor      0         21         1
  virginica       0          2        15
> # Confusion Matrix
> confusionMatrix(cm)
Confusion Matrix and Statistics

            predicted
             setosa versicolor virginica
  setosa         18          0         0
  versicolor      0         21         1
  virginica       0          2        15

Overall Statistics

               Accuracy : 0.9474
                 95% CI : (0.8538, 0.989)
    No Information Rate : 0.4035
    P-Value [Acc > NIR] : < 2.2e-16

                  Kappa : 0.9204

 Mcnemar's Test P-Value : NA

Statistics by Class:

                     Class: setosa Class: versicolor Class: virginica
Sensitivity                 1.0000            0.9130           0.9375
Specificity                 1.0000            0.9706           0.9512
Pos Pred Value              1.0000            0.9545           0.8824
Neg Pred Value              1.0000            0.9429           0.9750
Prevalence                  0.3158            0.4035           0.2807
Detection Rate              0.3158            0.3684           0.2632
Detection Prevalence        0.3158            0.3860           0.2982
Balanced Accuracy           1.0000            0.9418           0.9444
>
```

Practical 5

The dataset iris contains 50 samples from each of 3 species of Iris (Iris setosa, Iris virginica and Iris versicolor).

Exercise A

Split the data to training and test data. Build a KNN model for this data with different 'k' values

Solution

```
> # Practical 5
> # Exercise A
> # Build KNN for iris dataset
> # Loading packages
> library(e1071)
> library(caTools)
> library(class)
>
> # Loading data
> data(iris)
> head(iris)
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1          5.1         3.5          1.4         0.2  setosa
2          4.9         3.0          1.4         0.2  setosa
3          4.7         3.2          1.3         0.2  setosa
4          4.6         3.1          1.5         0.2  setosa
5          5.0         3.6          1.4         0.2  setosa
6          5.4         3.9          1.7         0.4  setosa
>
> # Splitting data into train and test data
> split = sample.split(iris$Sepal.Length, SplitRatio = 0.6)
> train_class = subset(iris, split == TRUE)
> test_class = subset(iris, split == FALSE)
>
> # Scaling Features
> train_scale = scale(train_class[, 1:4])
> test_scale = scale(test_class[, 1:4])
>
> # Building KNN Classifier for different 'k' values
> SpeciesClassifierKNN_01 = knn(train = train_scale, test = test_scale, cl = train_class$Species, k = 1)
> SpeciesClassifierKNN_01
 [1] setosa     setosa     setosa     setosa     setosa     setosa     setosa     setosa     setosa     setosa     setosa     setosa     setosa
[14] setosa     setosa     setosa     setosa     setosa     setosa     versicolor versicolor versicolor versicolor versicolor versicolor versicolor
[27] versicolor versicolor versicolor versicolor versicolor versicolor versicolor versicolor versicolor virginica  virginica  versicolor virginica
[40] virginica  virginica  virginica  virginica  virginica  versicolor virginica  virginica  virginica  virginica  versicolor virginica  virginica
[53] versicolor virginica  virginica  virginica  virginica
Levels: setosa versicolor virginica
>
> SpeciesClassifierKNN_03 = knn(train = train_scale, test = test_scale, cl = train_class$Species, k = 3)
> SpeciesClassifierKNN_03
 [1] setosa     setosa     setosa     setosa     setosa     setosa     setosa     setosa     setosa     setosa     setosa     setosa     setosa
[14] setosa     setosa     setosa     setosa     setosa     setosa     versicolor versicolor versicolor versicolor versicolor versicolor versicolor
[27] versicolor versicolor versicolor versicolor versicolor versicolor versicolor versicolor versicolor virginica  virginica  versicolor virginica
[40] virginica  virginica  virginica  virginica  virginica  versicolor virginica  virginica  virginica  versicolor versicolor virginica  virginica
[53] versicolor virginica  virginica  virginica  virginica
Levels: setosa versicolor virginica
>
> SpeciesClassifierKNN_15 = knn(train = train_scale, test = test_scale, cl = train_class$Species, k = 15)
> SpeciesClassifierKNN_15
 [1] setosa     setosa     setosa     setosa     setosa     setosa     setosa     setosa     setosa     setosa     setosa     setosa     setosa
[14] setosa     setosa     setosa     setosa     setosa     setosa     versicolor versicolor versicolor versicolor versicolor versicolor versicolor
[27] versicolor versicolor versicolor versicolor versicolor versicolor versicolor versicolor versicolor virginica  virginica  versicolor virginica
[40] virginica  virginica  virginica  virginica  virginica  versicolor virginica  virginica  versicolor versicolor versicolor virginica  versicolor
[53] versicolor virginica  virginica  virginica  virginica
Levels: setosa versicolor virginica
>
```

Exercise B

Build the confusion matrix and calculate the accuracy for all ‘k’ values.

Solution

```
> # Exercise B
> # Build confusion matrix and calculate accuracy for all the models
> library(caret)
> # Building Confusion matrix for k = 1
> cm = table(test_class$Species, SpeciesClassifierKNN_01)
> cm
            SpeciesClassifierKNN_01
             setosa versicolor virginica
  setosa         19          0         0
  versicolor      0         16         0
  virginica       0          4        18
> confusionMatrix(cm)
Confusion Matrix and Statistics

            SpeciesClassifierKNN_01
             setosa versicolor virginica
  setosa         19          0         0
  versicolor      0         16         0
  virginica       0          4        18

Overall Statistics

               Accuracy : 0.9298
                 95% CI : (0.83, 0.9805)
    No Information Rate : 0.3509
    P-Value [Acc > NIR] : < 2.2e-16

                  Kappa : 0.895

 Mcnemar's Test P-Value : NA

Statistics by Class:

                     Class: setosa Class: versicolor Class: virginica
Sensitivity                 1.0000            0.8000           1.0000
Specificity                 1.0000            1.0000           0.8974
Pos Pred Value              1.0000            1.0000           0.8182
Neg Pred Value              1.0000            0.9024           1.0000
Prevalence                  0.3333            0.3509           0.3158
Detection Rate              0.3333            0.2807           0.3158
Detection Prevalence        0.3333            0.2807           0.3860
Balanced Accuracy           1.0000            0.9000           0.9487
>
> # Calculate accuracy
> error = mean(SpeciesClassifierKNN_01 != test_class$Species)
> print(paste('Accuracy k01 =', 1 - error))
[1] "Accuracy k01 = 0.929824561403509"
>
> # Building Confusion matrix for k = 3
> cm = table(test_class$Species, SpeciesClassifierKNN_03)
> cm
            SpeciesClassifierKNN_03
             setosa versicolor virginica
  setosa         19          0         0
  versicolor      0         16         0
  virginica       0          5        17
> confusionMatrix(cm)
Confusion Matrix and Statistics

            SpeciesClassifierKNN_03
             setosa versicolor virginica
  setosa         19          0         0
  versicolor      0         16         0
  virginica       0          5        17

Overall Statistics

               Accuracy : 0.9123
                 95% CI : (0.807, 0.9709)
    No Information Rate : 0.3684
    P-Value [Acc > NIR] : < 2.2e-16

                  Kappa : 0.8691

 Mcnemar's Test P-Value : NA

Statistics by Class:

                     Class: setosa Class: versicolor Class: virginica
Sensitivity                 1.0000            0.7619           1.0000
Specificity                 1.0000            1.0000           0.8750
Pos Pred Value              1.0000            1.0000           0.7727
Neg Pred Value              1.0000            0.8780           1.0000
Prevalence                  0.3333            0.3684           0.2982
Detection Rate              0.3333            0.2807           0.2982
Detection Prevalence        0.3333            0.2807           0.3860
Balanced Accuracy           1.0000            0.8810           0.9375
>
> # Calculate accuracy
> error = mean(SpeciesClassifierKNN_03 != test_class$Species)
> print(paste('Accuracy k03 =', 1 - error))
[1] "Accuracy k03 = 0.912280701754386"
>
```

```
> # Building Confusion matrix for k = 15
> cm = table(test_class$Species, SpeciesClassifierKNN_15)
> cm
            SpeciesClassifierKNN_15
             setosa versicolor virginica
  setosa         19          0         0
  versicolor      0         16         0
  virginica       0          7        15
> confusionMatrix(cm)
Confusion Matrix and Statistics

            SpeciesClassifierKNN_15
             setosa versicolor virginica
  setosa         19          0         0
  versicolor      0         16         0
  virginica       0          7        15

Overall Statistics

               Accuracy : 0.8772
                 95% CI : (0.7632, 0.9492)
    No Information Rate : 0.4035
    P-Value [Acc > NIR] : 1.534e-13

                  Kappa : 0.8178

 Mcnemar's Test P-Value : NA

Statistics by Class:

                     Class: setosa Class: versicolor Class: virginica
Sensitivity                 1.0000            0.6957           1.0000
Specificity                 1.0000            1.0000           0.8333
Pos Pred Value              1.0000            1.0000           0.6818
Neg Pred Value              1.0000            0.8293           1.0000
Prevalence                  0.3333            0.4035           0.2632
Detection Rate              0.3333            0.2807           0.2632
Detection Prevalence        0.3333            0.2807           0.3860
Balanced Accuracy           1.0000            0.8478           0.9167
>
> # Calculate accuracy
> error = mean(SpeciesClassifierKNN_15 != test_class$Species)
> print(paste('Accuracy k15 =', 1 - error))
[1] "Accuracy k15 = 0.87719298245614"
>
```

Practical 6

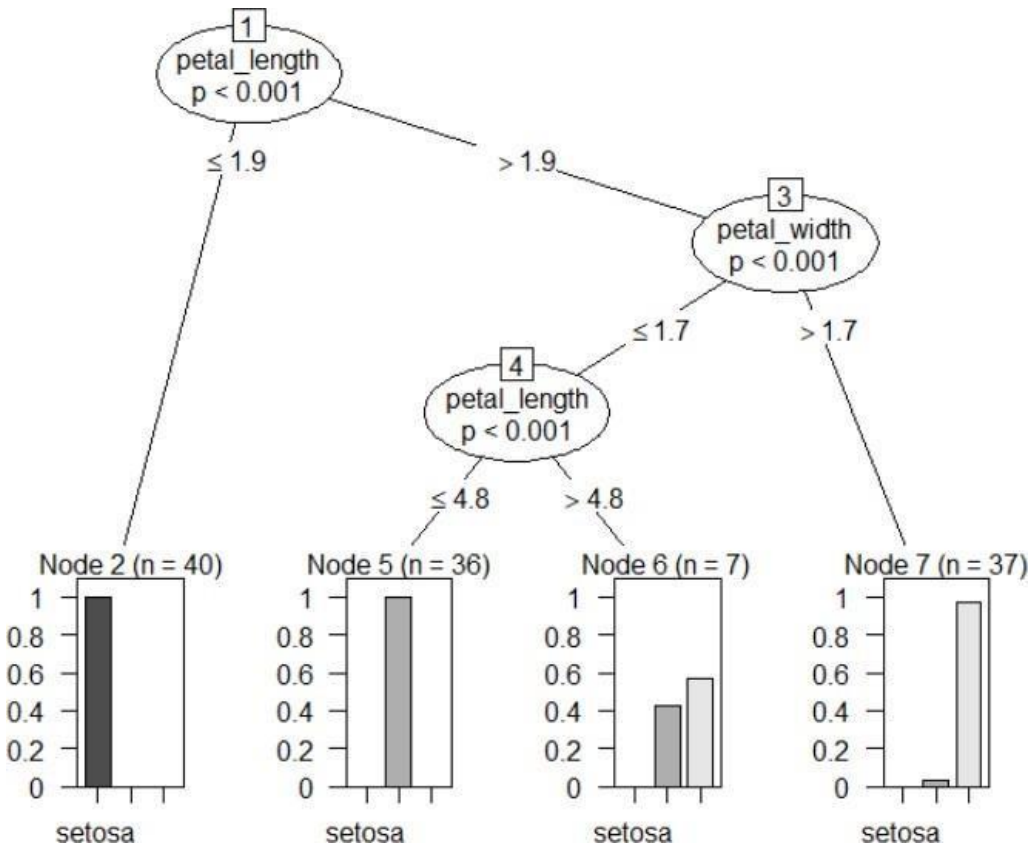
The dataset iris contains 50 samples from each of 3 species of Iris (Iris setosa, Iris virginica and Iris versicolor).

Exercise A

Split the data into training and test data. Build the decision tree for this data

Solution

```
> # Practical 6
> # Exercise A
> # Build a decision tree classifier for the iris dataset
> # Import required packages
> library(caTools)
> library(party)
> #Load iris dataset
> iris = read.csv("D:\\DataScience\\DM\\Datasets\\Iris.csv", stringsAsFactors = TRUE)
> # Splitting dataset into train and test data
> sample_data = sample.split(iris$species, SplitRatio = 0.8)
> train_data = subset(iris, sample_data == TRUE)
> test_data = subset(iris, sample_data == FALSE)
> # Model decision tree classifier
> model = ctree(species ~ ., train_data)
> plot(model)
>
```



Exercise B

Predict the species of the test data and determine the accuracy of the model

Solution

```
> # Exercise B
> # Predicting the test data
> predicted = predict(model, test_data)
> # Create a table to count how many are classified incorrectly
> predicted_vs_actual <- table(test_data$species, predicted)
> predicted_vs_actual
            predicted
             setosa versicolor virginica
  setosa         10          0         0
  versicolor      0          9         1
  virginica       0          1         9
> # Calculating the accuracy of the model
> sum(diag(m_at)) / sum(m_at)
[1] 0.9666667
>
```

Practical 7

The dataset iris contains 50 samples from each of 3 species of Iris (Iris setosa, Iris virginica and Iris versicolor).

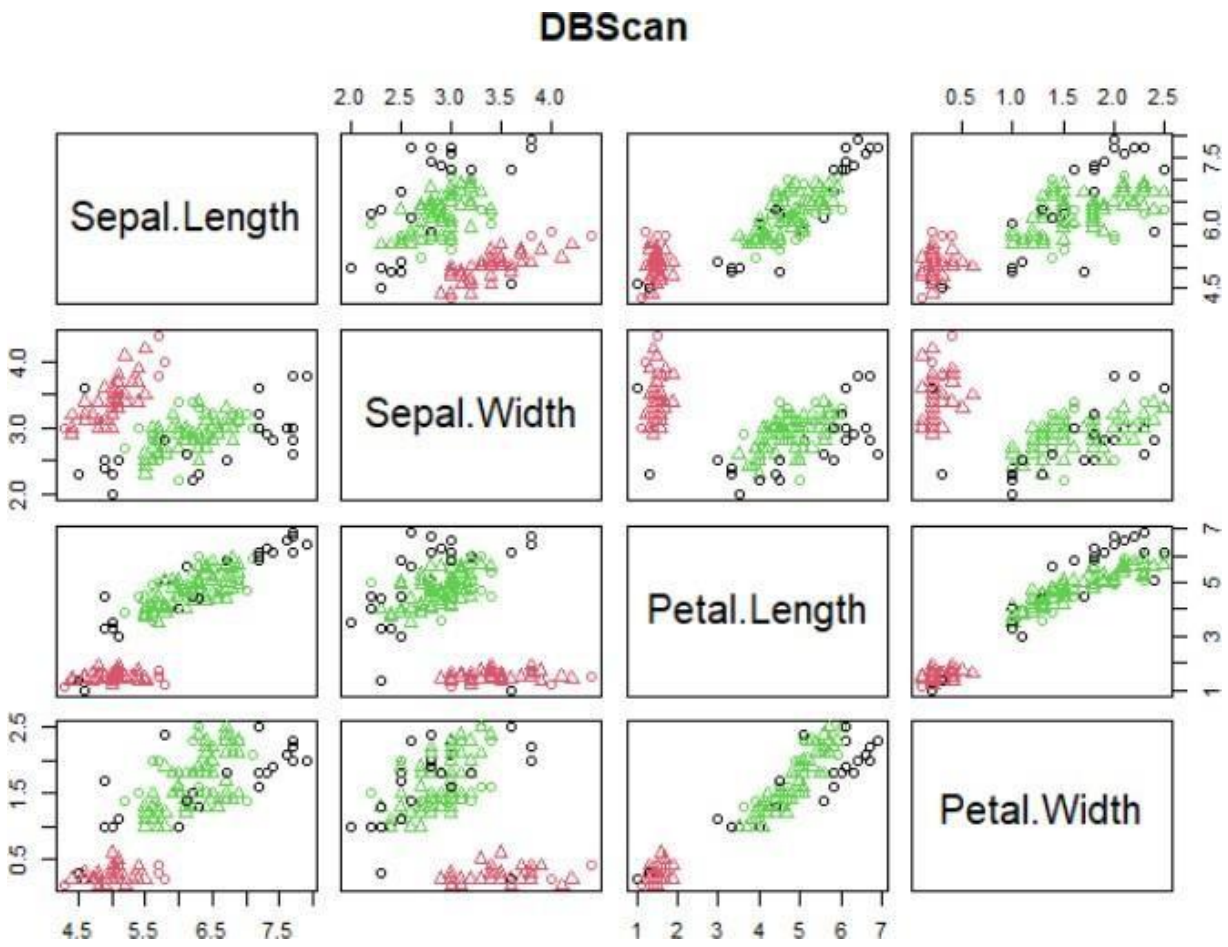
Exercise

Build DBScan clustering Model and plot it.

Solution

```
> # Practical 7
> # Build and plot DBScan model for iris dataset
> # Loading package
> library(fpc)
> data("iris")
> # Remove label form dataset
> iris_features = iris[-5]
> # Fitting DBScan clustering Model
> # to training dataset
> set.seed(220)  # Setting seed
> DBScanModel = dbscan(iris_features, eps = 0.45, MinPts = 5)
> DBScanModel
dbscan Pts=150 MinPts=5 eps=0.45
        0  1  2
border 24  4 13
seed    0 44 65
total  24 48 78
>
> # Display which record belong to which cluster
> DBScanModel$cluster
  [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 0 2
 [60] 2 0 2 0 2 2 2 2 2 0 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 0 2 2 2 2 2 0 2 2 2 2 0 2 2 2 2 2 2 0 0 0 0 0 2 2 2 2 0 2 2 0
[119] 0 2 2 2 0 2 2 0 2 2 2 0 0 0 2 2 0 0 2 2 2 2 2 2 2 2 2 2 2 2 2 2
>
> # Create a table to count how many are classified incorrectly
> table(DBScanModel$cluster, iris$Species)

    setosa versicolor virginica
  0      2          7        15
  1     48          0         0
  2      0         43        35
>
> # Plotting Cluster
> plot(DBScanModel, iris_features, main = "DBScan")
>
```


DBScan
Sepal.Length
Sepal.Width
Petal.Length
Petal.Width
2.0 2.5 3.0 3.5 4.0
0.5 1.0 1.5 2.0 2.5
4.5 6.0 7.5
2.0 3.0 4.0
1 3 5 7
0.5 1.5 2.5
4.5 5.5 6.5 7.5
1 2 3 4 5 6 7

Practical 8

The dataset iris mtcars(motor trend car road test) comprises fuel consumption, performance and 10 aspects of automobile design for 32 automobiles.

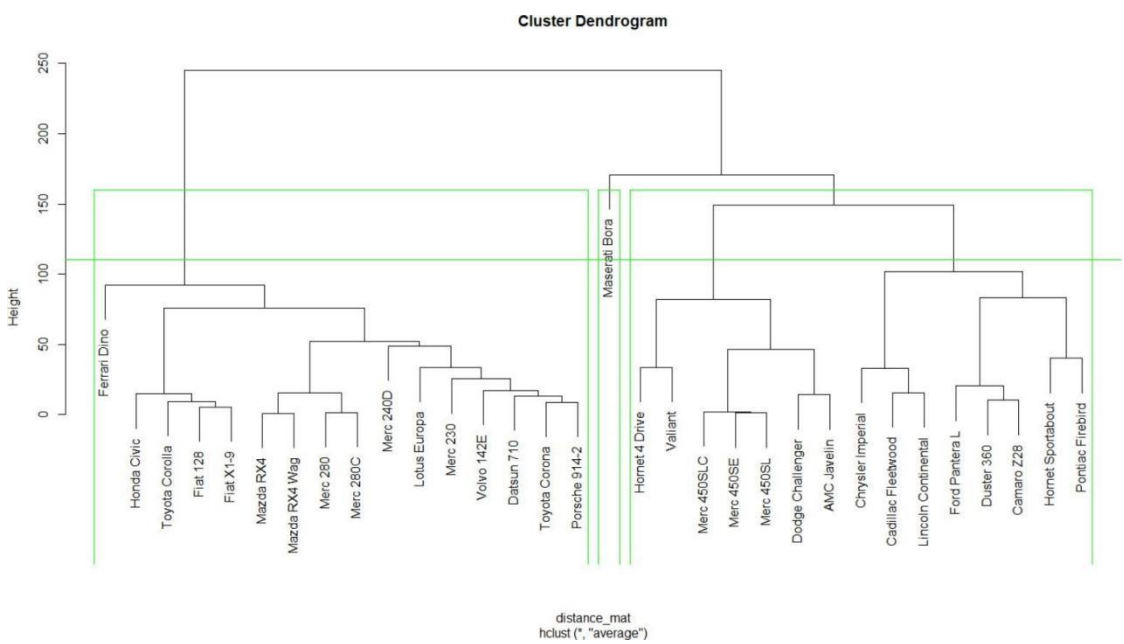
Description of the Dataset
A data frame with 32 observations on 11 (numeric) variables.

1. mpg - Miles/(US) gallon
2. cyl – Number of cylinders
3. disp – Displacement (cubic inches)
4. hp – Gross horsepower
5. drat – rear axle ratio
6. wt – Weight (1000 lbs)
7. qsec – ¼ mile time
8. vs - Engine(0 = V-shaped, 1 = straight)
9. am – Transmission (0 = automatic, 1 = manual)
10. gear – Number of forward gears
11. carb – Number of carburetors

Exercise

Perform hierarchical clustering and plot the dendrogram and cut the tree by the number of clusters.

Solution

```
> # Practical 9
> # Perform hierarchical clustering and plot the dendrogram
> # Import dataset
> data("mtcars")
> # Calculate distance matrix
> distance_mat = dist(mtcars, method = 'euclidean')
> distance_mat
                     Mazda RX4 Mazda RX4 Wag  Datsun 710 Hornet 4 Drive Hornet Sportabout     Valiant  Duster 360
Mazda RX4 Wag         0.6153251
Datsun 710           54.9086059    54.8915169
Hornet 4 Drive       98.1125212    98.0958939 150.9935191
Hornet Sportabout   210.3374396   210.3358546 265.0831615    121.0297564
Valiant              65.4717710    65.4392224 117.7547018     33.5508692       152.1241352
Duster 360          241.4076490   241.4088680 294.4790230    169.4299647        70.1767262 194.6094525
Merc 240D            50.1532711    50.1146059  49.6584796    121.2739722       241.5069657  89.5911056 281.2962502
Merc 230             25.4683117    25.3284509  33.1803843    118.2433145       233.4924012  85.0079649 265.8823313
Merc 280             15.3641921    15.2956865  66.9363534     91.4224033       199.3344960  60.2909811 227.8998521
Merc 280C            15.6724727    15.5837744  67.0261397     91.4612914       199.3406564  60.2655656 227.8813169
Merc 450SE          135.4307018   135.4254826 189.1954941     72.4964325        84.3888482  90.6970264 106.4084264
Merc 450SL          135.4014424   135.3960351 189.1631745     72.4313532        84.3683999  90.6769728 106.4320572
Merc 450SLC         135.4794674   135.4723157 189.2345426     72.5718466        84.4332423  90.7092989 106.4010305
Cadillac Fleetwood  326.3395903   326.3355070 381.0926242    234.4403876       116.2804201 266.6280942 119.0239068
Lincoln Continental 318.0469808   318.0429333 372.8012090    227.9726091       108.0624299 259.6304391 104.5112999
Chrysler Imperial   304.7203408   304.7169175 359.3014906    218.1548299        97.2049146 248.7713290  81.4297699
Fiat 128             93.2679950    93.2530993  40.9933763    184.9689734       302.0377212 152.1153263 333.9792070
```

```
AMC Javelin
Camaro Z28
Pontiac Firebird
Fiat X1-9
Porsche 914-2
Lotus Europa
Ford Pantera L
Ferrari Dino          224.4587490
Maserati Bora          86.9383253  223.5342175
Volvo 142E            277.4803312   70.4751034   289.1157363
> # Fitting Hierarchical clustering Model to sample dataset
> set.seed(240)
> Hierar_cl = hclust(distance_mat, method = "average")
> Hierar_cl

Call:
hclust(d = distance_mat, method = "average")

Cluster method   : average
Distance         : euclidean
Number of objects: 32

> # Plotting Dendrogram
> plot(Hierar_cl)
> # Cutting tree by height (on graph)
> abline(h = 110, col = "green")
> # Adding bounding box to the clusters
> rect.hclust(Hierar_cl, k = 3, border = "green")
> # Cutting tree by no. of clusters
> fit = cutree(Hierar_cl, k = 3 )
> fit
          Mazda RX4       Mazda RX4 Wag          Datsun 710      Hornet 4 Drive   Hornet Sportabout             Valiant
                  1                   1                   1                   2                   2                   2
         Duster 360           Merc 240D            Merc 230            Merc 280           Merc 280C          Merc 450SE
                  2                   1                   1                   1                   1                   2
         Merc 450SL          Merc 450SLC  Cadillac Fleetwood Lincoln Continental   Chrysler Imperial            Fiat 128
                  2                   2                   2                   2                   2                   1
        Honda Civic      Toyota Corolla       Toyota Corona    Dodge Challenger         AMC Javelin          Camaro Z28
                  1                   1                   1                   2                   2                   2
   Pontiac Firebird           Fiat X1-9       Porsche 914-2        Lotus Europa      Ford Pantera L        Ferrari Dino
                  2                   1                   1                   1                   2                   1
      Maserati Bora          Volvo 142E
                  3                   1
> table(fit)
fit
 1  2  3
16 15  1
>
```